

AI FAQ ASSISTANT API

AI FAQ ASSISTANT API

RESTful FAQ Management and AI-Powered Question Answering System.

Team ID: SWTID-2026-8964

Team leader: H. KALAIYARASI

Mail ID : kalaiyarasi01072007@gmail.com

S.No	Team Members Name	Email Id
1	GOPIKA.A	gopika2007007@gmail.com
2	GOTHAWARI.C	gothawari99@gmail.com
3	ISHWARIYA .M	aishwaryam364@gmail.com
4	LOKESHWARI .N	lokeshwari02122006@gmail.com

DESCRIPTION

The AI FAQ Assistant API is a secure and intelligent RESTful backend application designed to simplify Frequently Asked Questions (FAQ) management and provide AI-powered question answering services. The system is developed using Node.js and Express.js and uses MongoDB with Mongoose for data storage and schema management.

The application provides secure user registration and login using JWT authentication. Passwords are protected using bcrypt hashing before they are stored in the database. Authenticated users can create, view, update, and delete their own FAQ records. Public users can retrieve and search FAQ information without authentication.

The system is integrated with Google Gemini AI using the official "@google/genai" SDK. The AI service provides two major functions: generating direct answers for user questions and generating structured FAQ question-and-answer pairs from a given topic.

The application follows a modular backend architecture containing configuration, controllers, middleware, models, routes, services, utilities, and server files. Centralized error handling and request validation improve reliability and maintainability.

The project can be tested using Postman, and an automated integration test file is provided to verify authentication, FAQ operations, search functionality, AI services, and error handling.

SCENARIO-BASED CASE STUDY

Background

A student, employee, organization, or customer may need quick answers to frequently asked questions. In a traditional FAQ system, questions and answers are manually created and maintained. Searching for a particular answer can also become difficult when the number of FAQ records increases.

At the same time, preparing new FAQ questions and answers manually requires additional time. Users may also need an intelligent system that can provide answers to questions that are not already stored in the FAQ database.

The AI FAQ Assistant API is developed to solve these problems through a centralized backend system that combines FAQ management, authentication, search, database storage, and Google Gemini AI services.

Problem

The existing manual FAQ management process may have the following problems:

- FAQ information may be difficult to organize and maintain.
- Creating large numbers of question-and-answer pairs manually requires more time.
- Users may need to search through many FAQ records to find the required information.
- There is no intelligent answer generation for questions that are not already available as FAQ records.
- Unauthenticated users should not be allowed to create, modify, or delete FAQ records.
- Passwords must be protected instead of being stored directly.
- API errors should be handled in a consistent manner.
- Database validation is required to maintain correct FAQ and user information.

Solution

The AI FAQ Assistant API provides a centralized RESTful backend solution for managing FAQ information and providing AI-powered assistance.

The system uses Express.js to create API endpoints, MongoDB and Mongoose to store and validate data, JWT to protect private endpoints, and bcrypt to securely hash passwords.

Google Gemini AI is integrated through the "@google/genai" package to provide intelligent answers and automatically generate structured FAQ question-and-answer pairs.

Usage

Authenticated User Usage

- Register a new account.
- Login using email and password.
- Receive a JWT authentication token.
- Access the protected user profile endpoint.
- Create FAQ records.
- Update FAQ records created by the user.
- Delete FAQ records created by the user.
- Generate AI answers.
- Generate FAQ question-and-answer pairs using AI.

Public User Usage

- View all available FAQs.
- View an individual FAQ using its ID.
- Search FAQs using a keyword.

Public users cannot access protected operations such as creating, updating, or deleting FAQ records.

Outcome

The AI FAQ Assistant API provides a centralized backend for FAQ management and AI-based assistance. Authentication protects private operations, bcrypt protects passwords, Mongoose provides database validation, and Google Gemini provides intelligent answer and FAQ generation capabilities.

The system also provides automated integration testing to verify the major API operations.

1.SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11 or another operating system capable of running Node.js and MongoDB.
- **Node.js:** Node.js runtime is required to execute the backend JavaScript application.
- **npm:** Node Package Manager is used to install and manage project dependencies.
- **Express.js:** Used to create the RESTful API server and route HTTP requests.
- **MongoDB:** NoSQL database used to store users and FAQ records.
- **Mongoose:** ODM library used for MongoDB connection, schema definition, validation, and database operations.
- **Google Gemini AI:** Used for AI-based answer generation and FAQ generation.
- **bcrypt:** Used for password hashing.
- **JSON Web Token:** Used to authenticate users and protect private API routes.
- **CORS:** Used to manage Cross-Origin Resource Sharing.
- **dotenv:** Used to load environment variables from the ".env" file.
- **Postman:** Used to test REST API endpoints.
- **Code Editor:** Any suitable JavaScript code editor can be used.

2.HARDWARE REQUIREMENTS

- **Processor:** Intel or AMD processor capable of running Node.js and MongoDB.
- **RAM:** Minimum 4 GB recommended; 8 GB provides a more comfortable development environment.

- **Storage:** Sufficient free disk space for Node.js, project dependencies, MongoDB data, and project files.
- **Network:** Internet connection is required when accessing the Google Gemini API.
- **Input Device:** Keyboard and mouse.
- **Display:** Standard monitor or laptop display.

EPIC-1: PROJECT ARCHITECTURE

The AI FAQ Assistant API follows a modular backend architecture. Incoming HTTP requests are received by the Express.js application and passed through appropriate routes, middleware, controllers, services, and database operations.

The major components of the system are described below.

Components Definition

1.API Client / Postman

Postman is used as the client interface for sending HTTP requests to the API.

The client can send:

- GET requests
- POST requests
- PUT requests
- DELETE requests

The API returns responses in JSON format.

2.Express Server

Express.js provides the main REST API framework.

It is responsible for:

- Creating API routes.
- Receiving HTTP requests.

- Processing JSON request bodies.
- Connecting routes with controllers.
- Applying middleware.
- Returning JSON responses.

3.Authentication Middleware

The authentication middleware verifies JWT tokens for protected routes.

The middleware:

- Reads the Authorization header.
- Extracts the Bearer token.
- Verifies the JWT.
- Identifies the authenticated user.
- Allows the request to continue when the token is valid.
- Rejects unauthorized requests when the token is missing or invalid.

4.Controller Layer

Controllers contain the main application logic.

The project contains:

- "authController.js"
- "faqController.js"
- "aiController.js"

The authentication controller manages registration, login, and profile operations.

The FAQ controller manages FAQ creation, retrieval, update, deletion, and search operations.

The AI controller communicates with the Gemini service for AI operations.

5.Service Layer

The "geminiService.js" file manages communication with the Google Gemini AI service.

It provides the application with AI-powered:

- Question answering.
- FAQ generation.

6.Database Layer

MongoDB is used as the database.

Mongoose provides:

- Schema definitions.
- Data validation.
- Database connection.
- CRUD operations.
- MongoDB document handling.

7.Error Handling Layer

The centralized error middleware processes application errors and returns structured error responses.

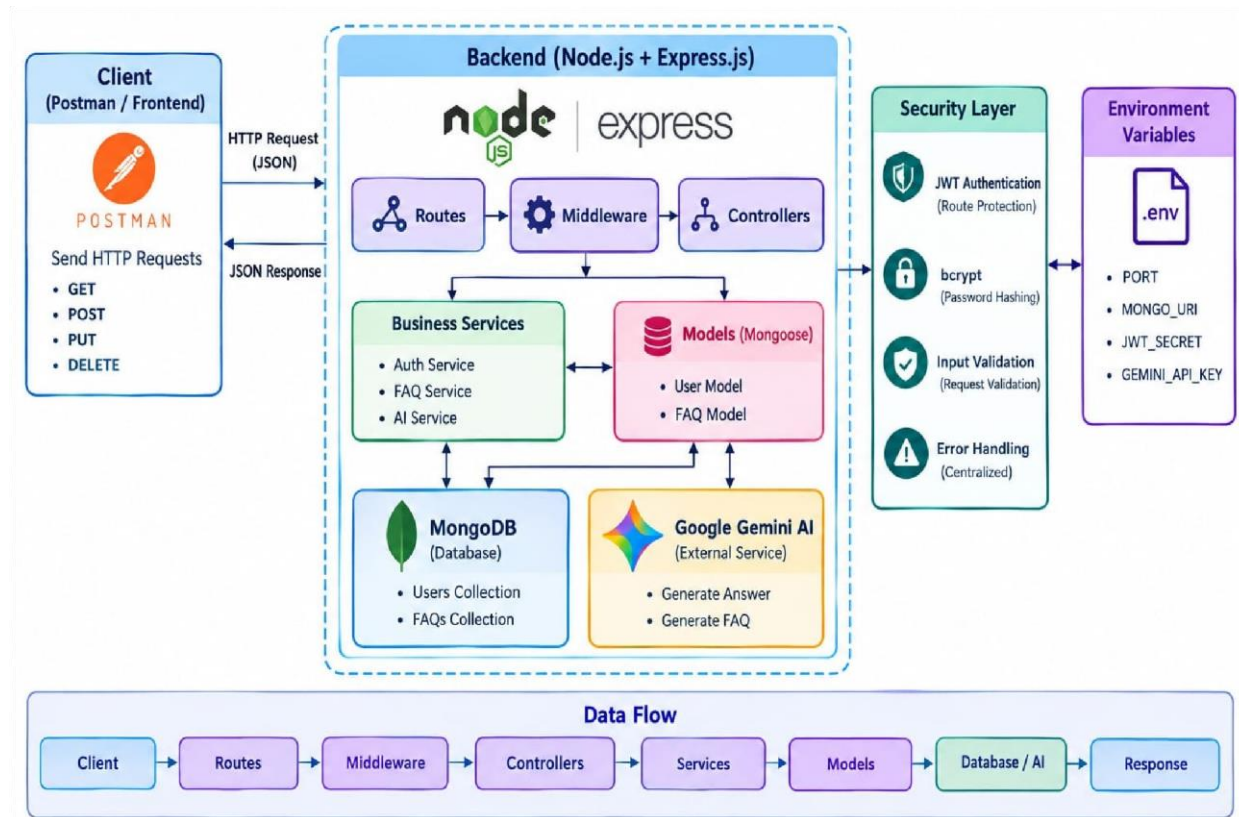
It handles situations such as:

- Validation errors.
- Invalid MongoDB IDs.
- Duplicate key errors.
- Database errors.
- General server errors.

8.AI Integration

Google Gemini AI is integrated through the official "@google/genai" package.

The AI layer receives user questions or topics and returns generated content.



SYSTEM ARCHITECTURE DIAGRAM

ENTITY-RELATIONSHIP / DATABASE STRUCTURE DESCRIPTION

The application uses MongoDB, which stores data as documents inside collections.

The main database entities are:

1. User Entity

The User collection stores registered user information.

Important fields include:

- "_id" – Unique MongoDB ObjectId.
- "name" – Name of the user.
- "email" – User email address.
- "password" – Hashed user password.

2. FAQ Entity

The FAQ collection stores question-and-answer information.

Important fields include:

- "_id" – Unique FAQ identifier.
- "question" – FAQ question.
- "answer" – FAQ answer.
- "category" – FAQ category.
- "createdBy" – Reference to the user who created the FAQ.

FAQ Categories

The application validates FAQ categories using the following categories:

- Technology
- Education
- Health
- Banking
- General

3.AI Generated Content

AI-generated FAQ content is generated dynamically by the Gemini service.

The generated content contains:

- Question
- Answer

The AI-generated result can be used as FAQ content and stored through the FAQ creation endpoint.

KEY STRUCTURAL MAPPINGS

User → FAQ

One user can create multiple FAQ records.

FAQ → User

Each FAQ stores the creator reference so that ownership can be verified for protected update and delete operations.

User → Authentication

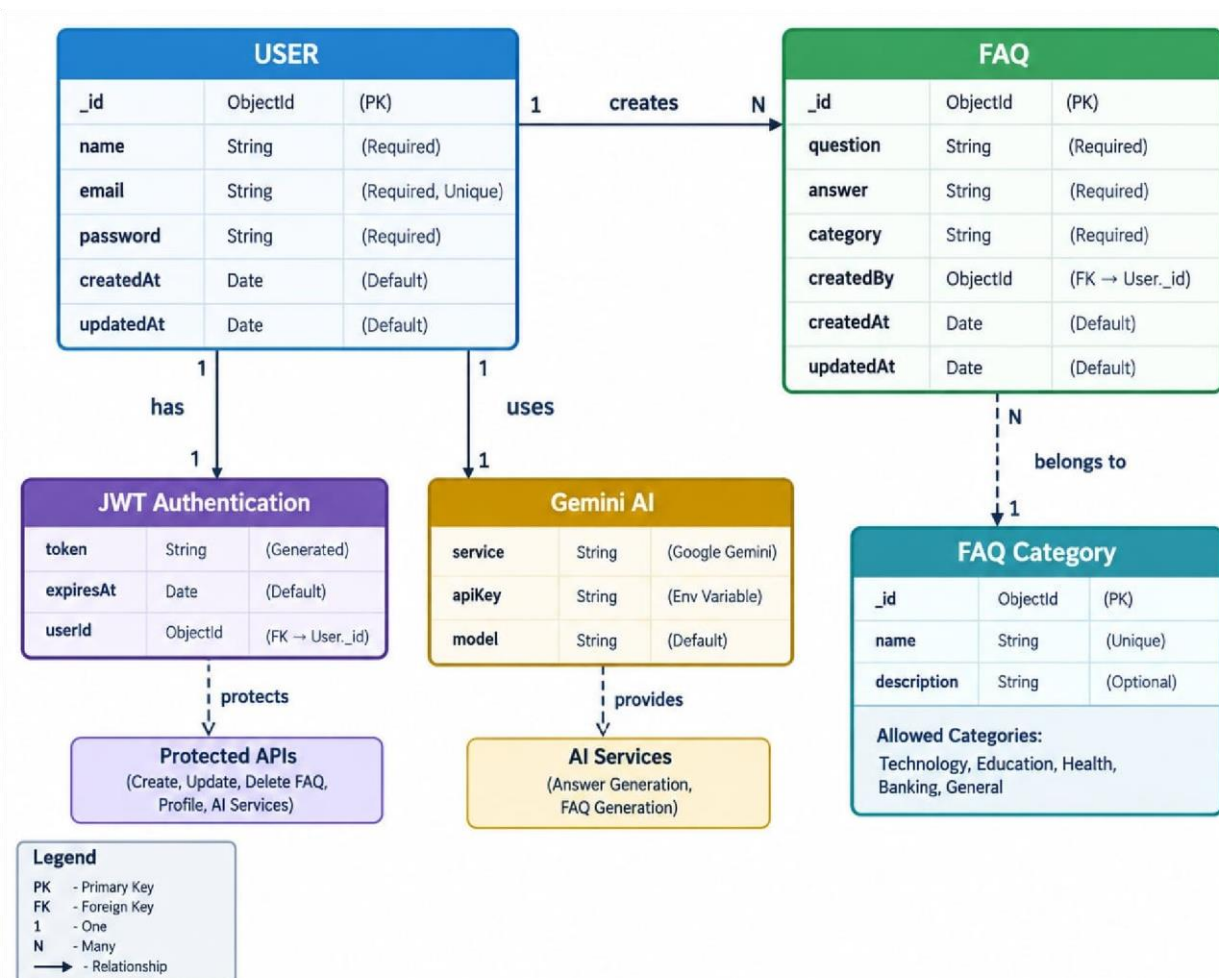
A registered user can login and receive a JWT token.

User → AI Services

An authenticated user can use the protected AI endpoints.

FAQ → Search

The search endpoint retrieves FAQ records based on the supplied keyword.



ENTITY RELATIONSHIP

KEY FEATURES

- User Registration: Allows new users to create an account.
- User Login: Authenticates users using email and password.
- JWT Authentication: Protects private API endpoints.
- Password Hashing: bcrypt is used to protect stored passwords.
- User Profile: Authenticated users can access their profile.
- FAQ Creation: Authenticated users can create FAQ records.
- FAQ Retrieval: Public users can retrieve FAQ information.
- FAQ Update: FAQ creators can update their own FAQ records.
- FAQ Delete: FAQ creators can delete their own FAQ records.
- FAQ Search: Users can search FAQ information using keywords.
- AI Answer Generator: Generates an answer for a user question.
- AI FAQ Generator: Generates a structured question-and-answer pair from a topic.
- Category Validation: FAQ categories are restricted to valid categories.
- Mongoose Validation: Maintains database data integrity.

ROLES AND RESPONSIBILITIES

Public User ●

View all FAQs.

- View a specific FAQ.
- Search FAQs.
- Cannot create FAQ records.
- Cannot update FAQ records.
- Cannot delete FAQ records.

- Cannot access protected AI endpoints.

Authenticated User

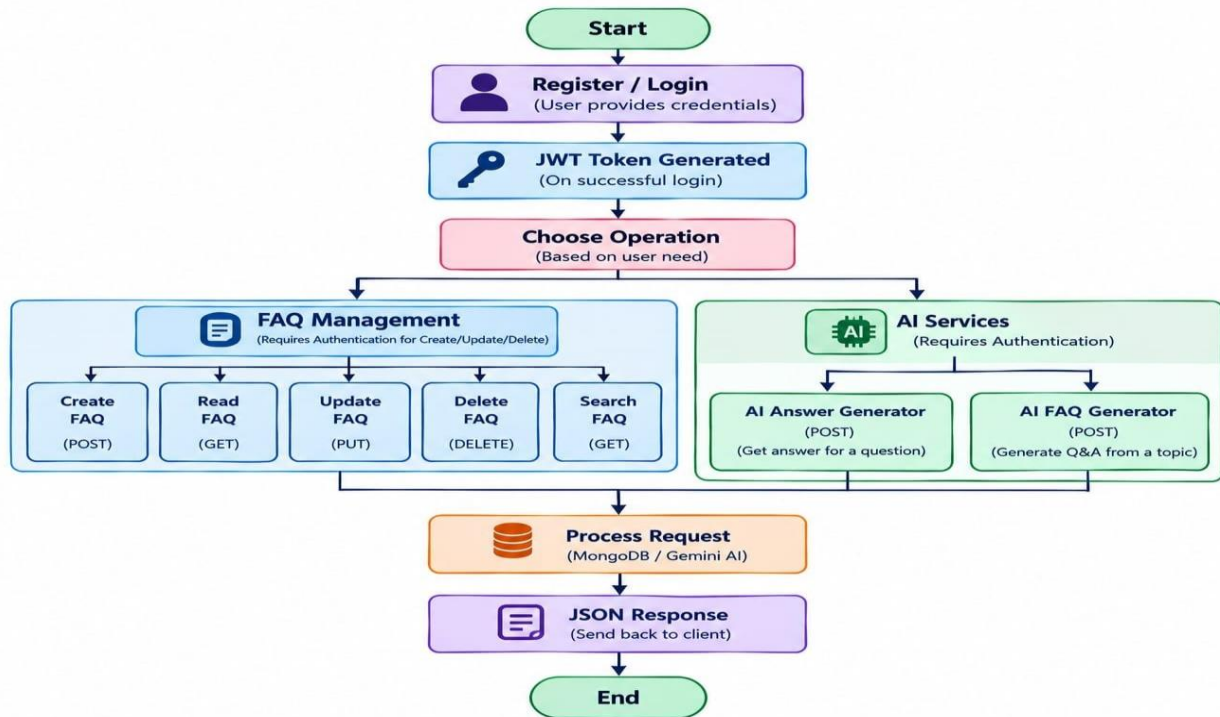
- Register and login.
- Access profile information.
- Create FAQs.
- Update own FAQ records.
- Delete own FAQ records.
- Search and view FAQs.
- Use AI Answer Generator.
- Use AI FAQ Generator.

USER FLOW

The general system flow is:

1. User sends a request through Postman or another API client.
2. Express.js receives the request.
3. The request is mapped to the appropriate route.
4. Public routes are processed directly.
5. Protected routes pass through JWT authentication middleware.
6. The controller processes the request.
7. The controller communicates with MongoDB or the Gemini AI service.
8. The result is returned as a JSON response.

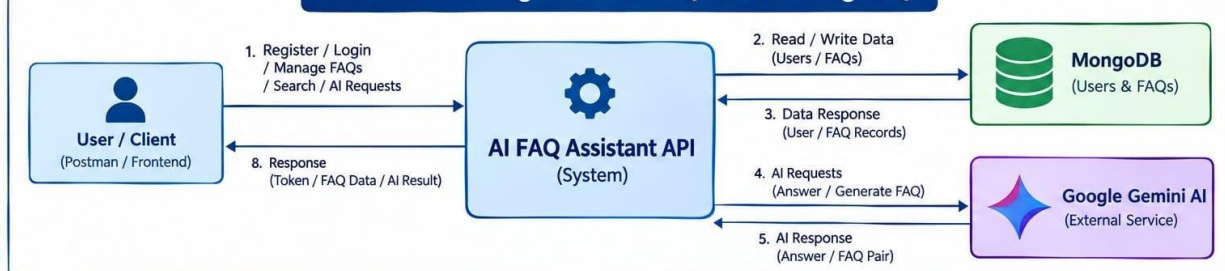
AI FAQ Assistant API – User Flow Diagram



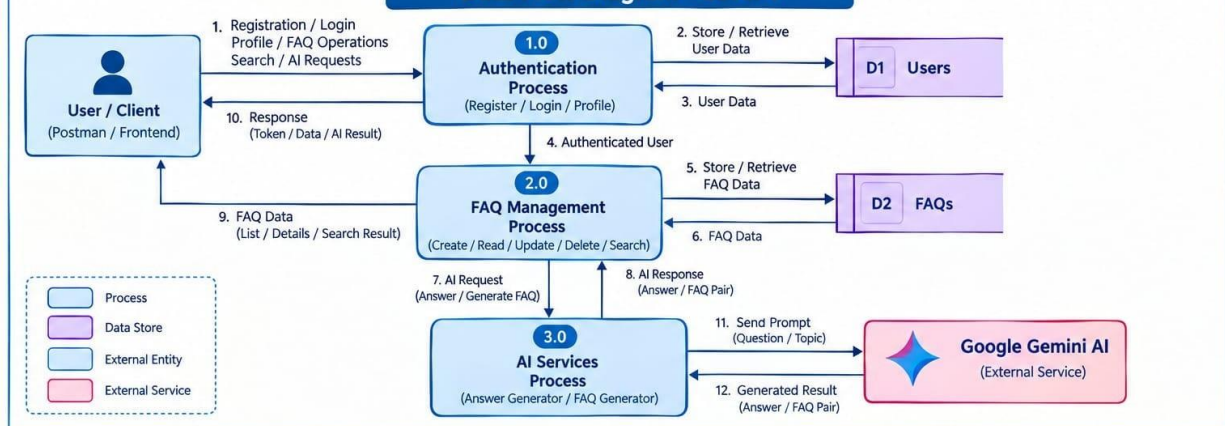
USER FLOW DIAGRAM

DATA FLOW DIAGRAM

Data Flow Diagram – Level 0 (Context Diagram)



Data Flow Diagram – Level 1



MVC ARCHITECTURE PATTERN

The project follows a modular architecture inspired by the MVC pattern.

Since the application is a backend API, there are no traditional HTML template views.

The API routes act as the interface through which clients communicate with the backend.

Model Layer

The Model layer defines the structure of data stored in MongoDB.

The main models are:

- "User.js"
- "FAQ.js"

Mongoose schemas define the fields and validation rules.

Controller Layer

The Controller layer handles application operations.

authController.js Handles:

- User registration.
- User login.
- User profile retrieval.

faqController.js

Handles:

- FAQ creation.
- FAQ retrieval.
- FAQ update.
- FAQ deletion.
- FAQ search.

Handles:

- AI answer generation.
- AI FAQ generation.

Service Layer

The "geminiService.js" file provides the connection between the application and Google Gemini AI.

Route Layer

Routes map HTTP requests to controllers.

The main route files are:

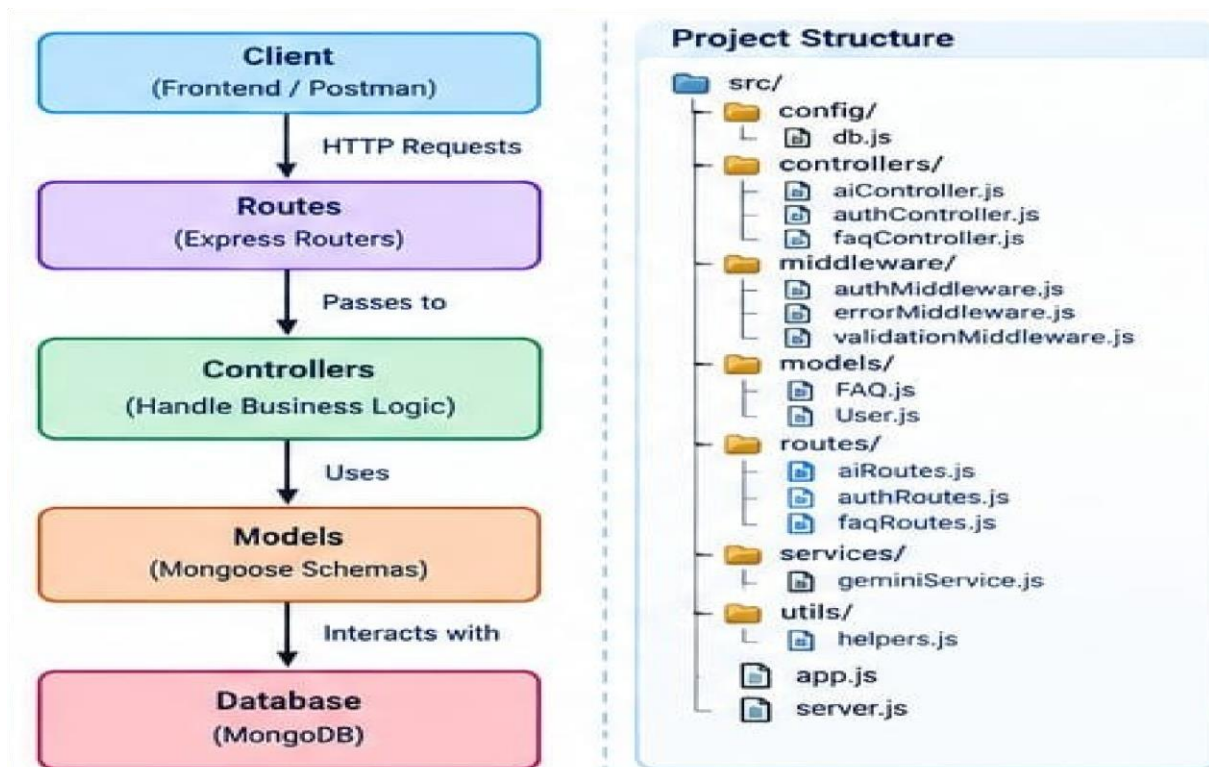
- "authRoutes.js"
- "faqRoutes.js"
- "aiRoutes.js"

Middleware Layer

Middleware provides additional processing for incoming requests.

Main middleware files are:

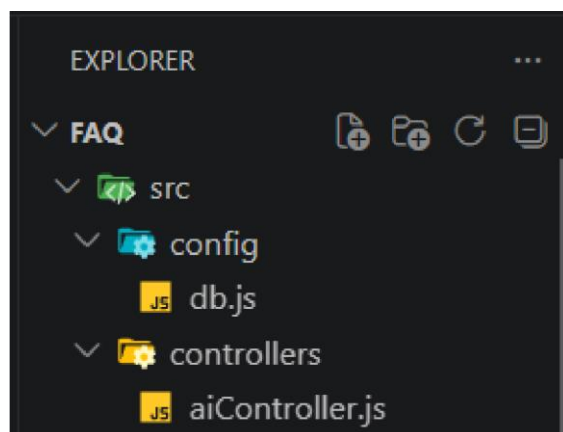
- "authMiddleware.js"
- "errorMiddleware.js"
- "validationMiddleware.js"



EPIC-2: PROJECT SETUP AND CONFIGURATION

Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.



Server Setup (npm init)

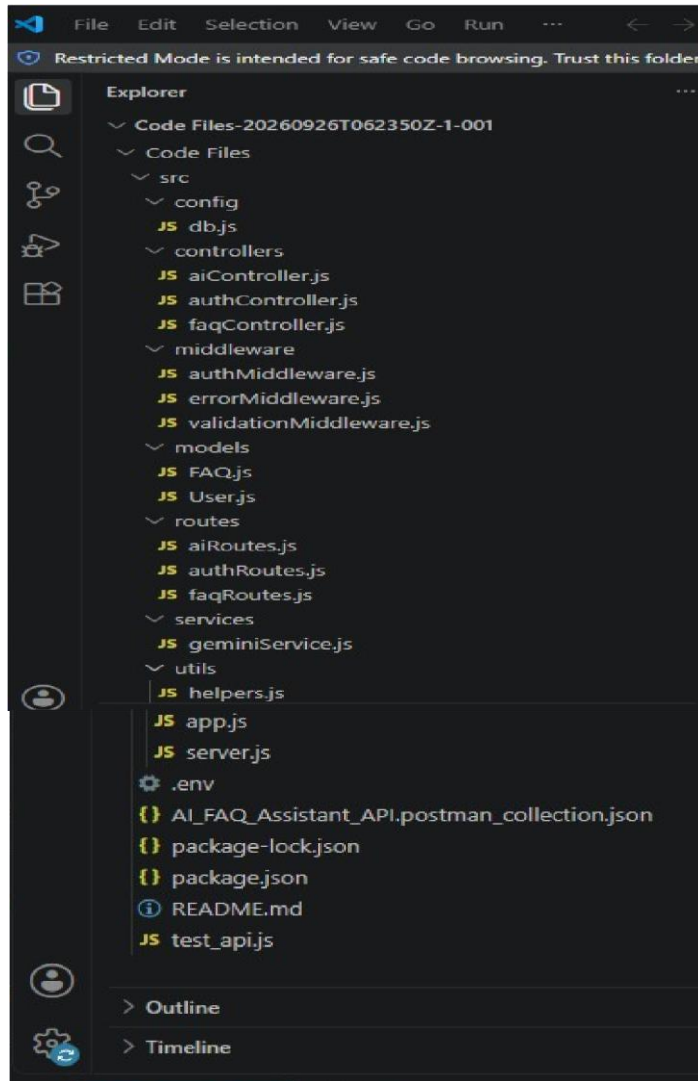
Open your root folder terminal in VS Code and run:

Bash

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcrypt
@google/genai
```

File and Folder Structure Matrix

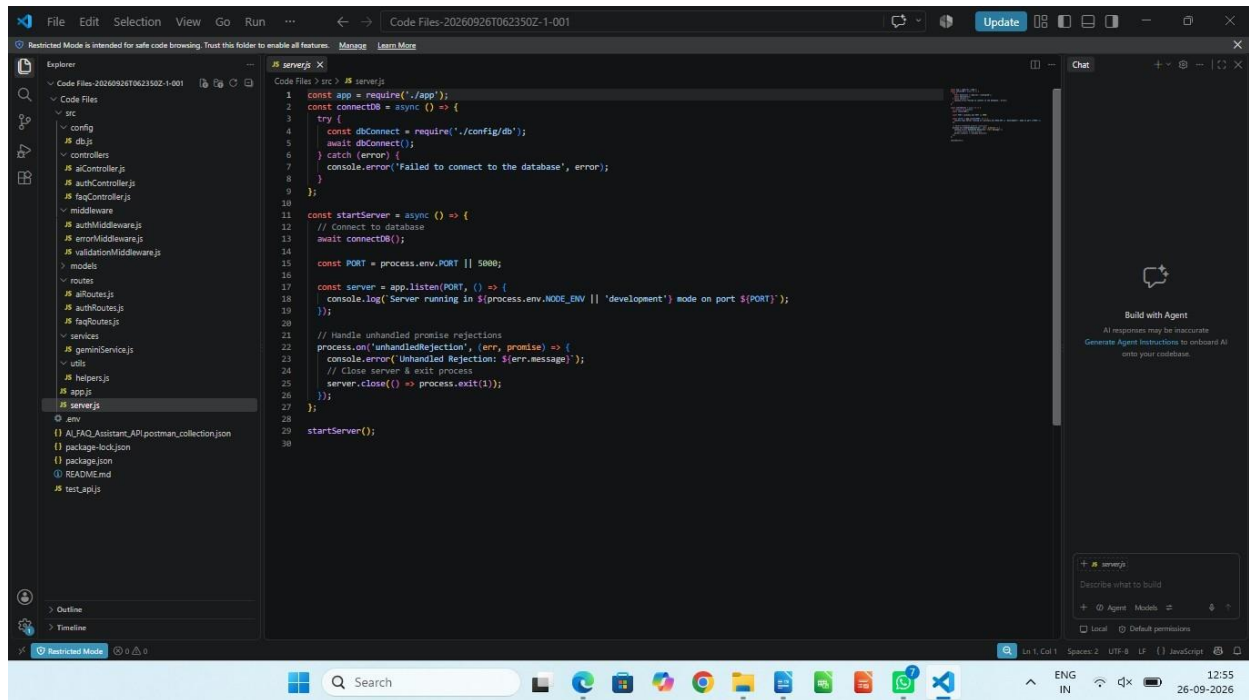
Plaintext



PROJECT FOLDER STRUCTURE EPIC-3: BACKEND DEVELOPMENT

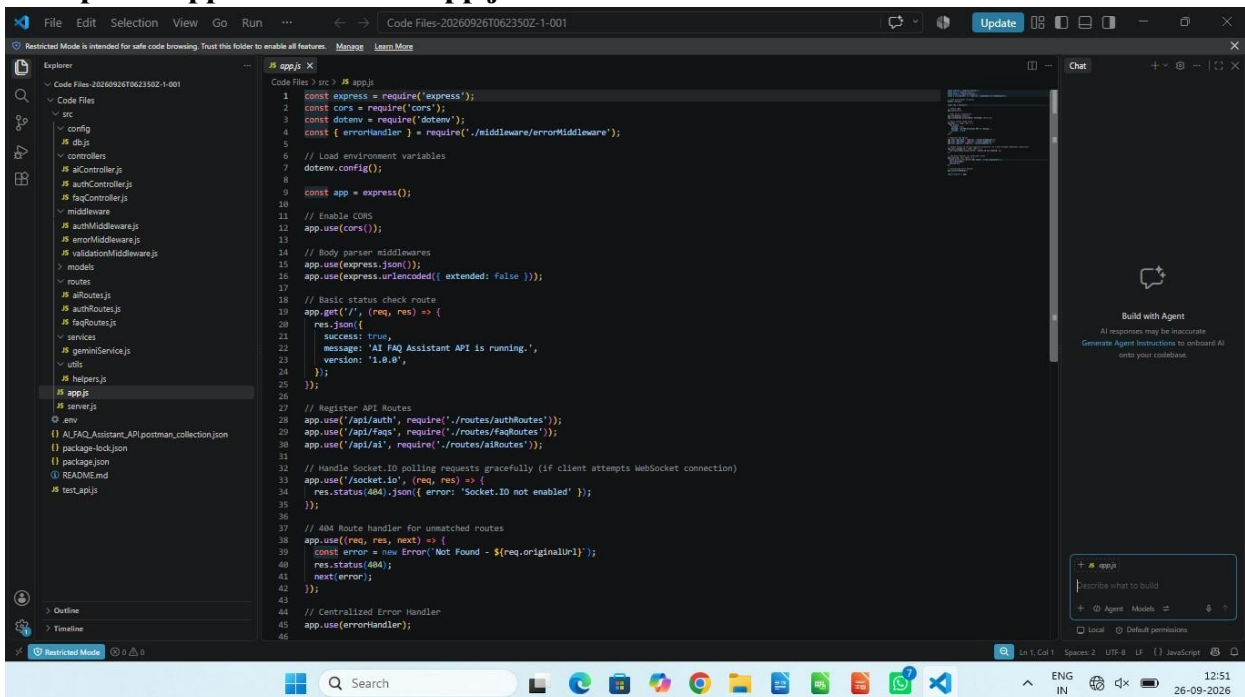
CORE SYSTEM SCRIPT IMPLEMENTATIONS

1. Gateway Server – src/server.js

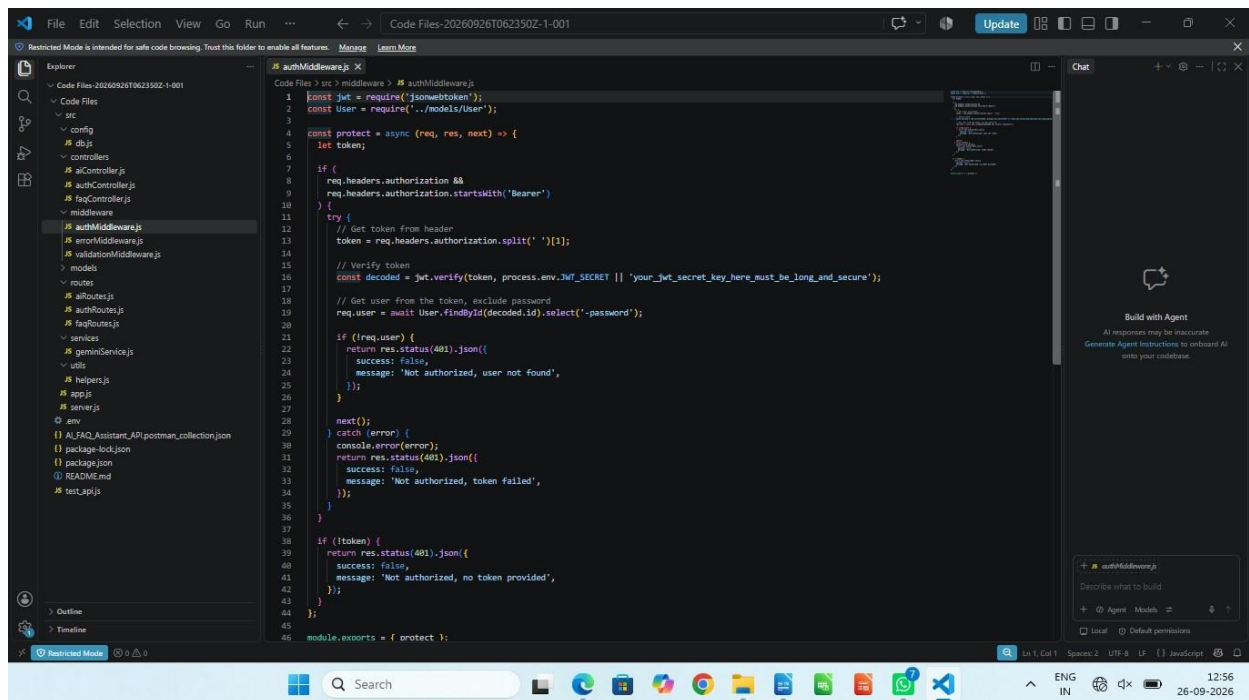


This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

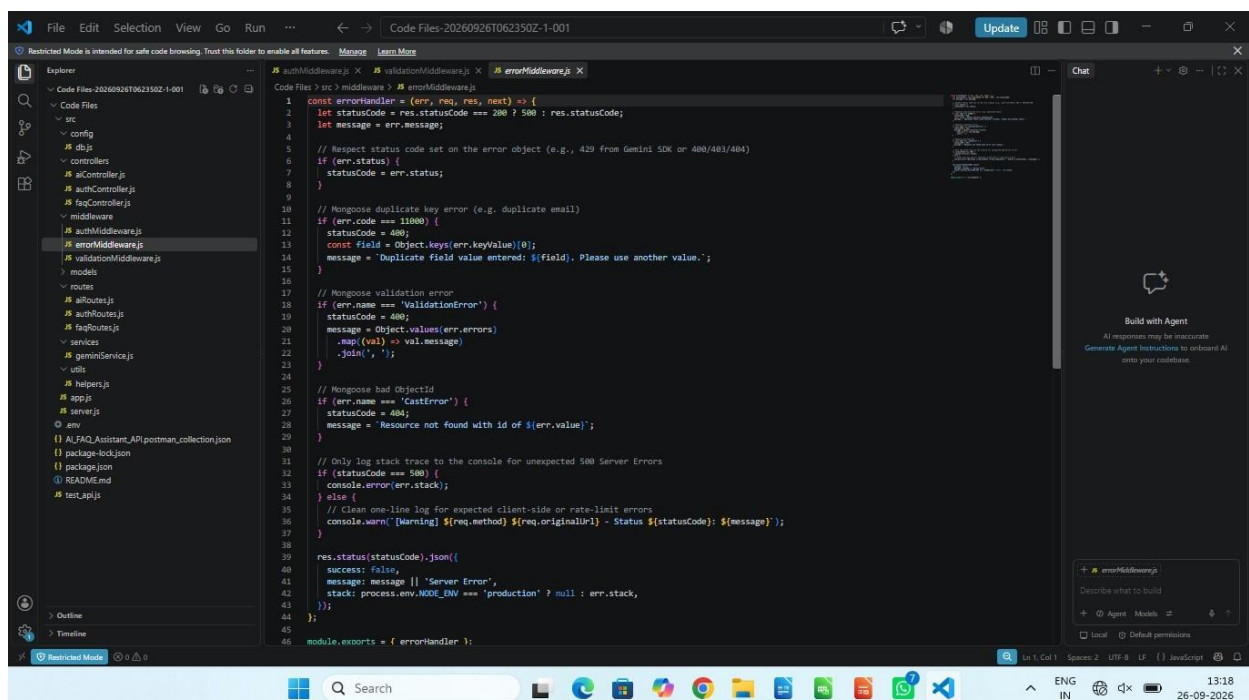
2.Express Application – src/app.js



3.Authentication Middleware



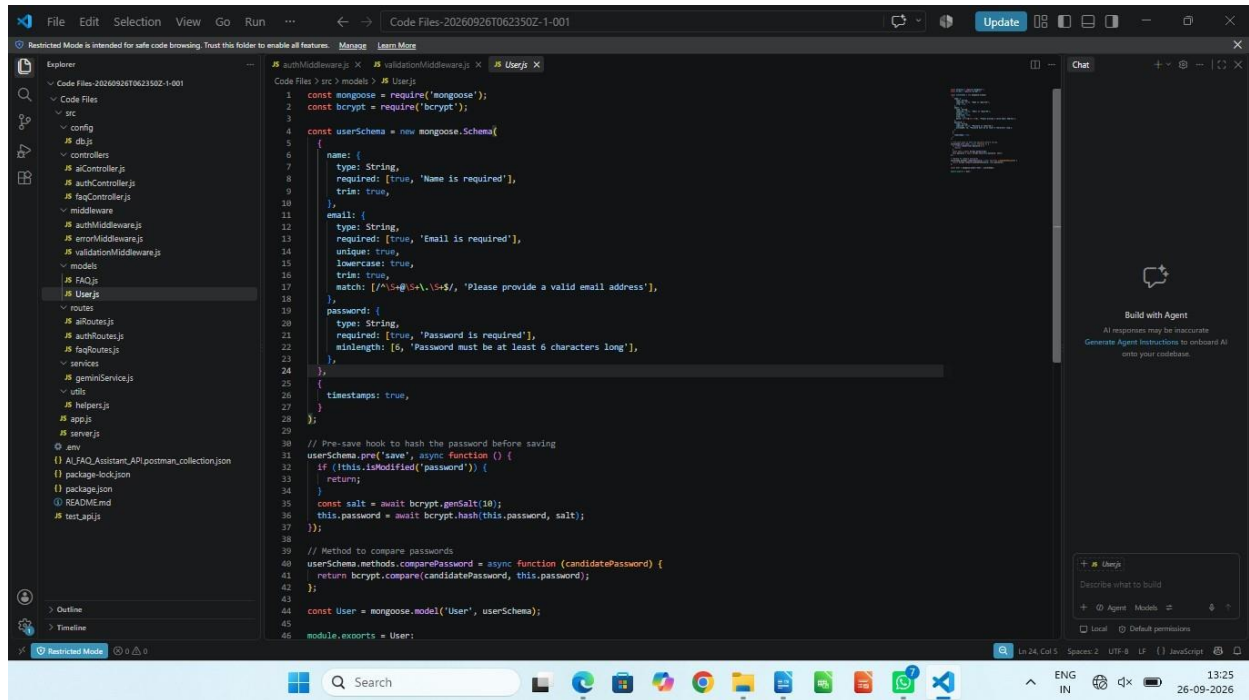
(src/middleware/errorMiddleware.js)



Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

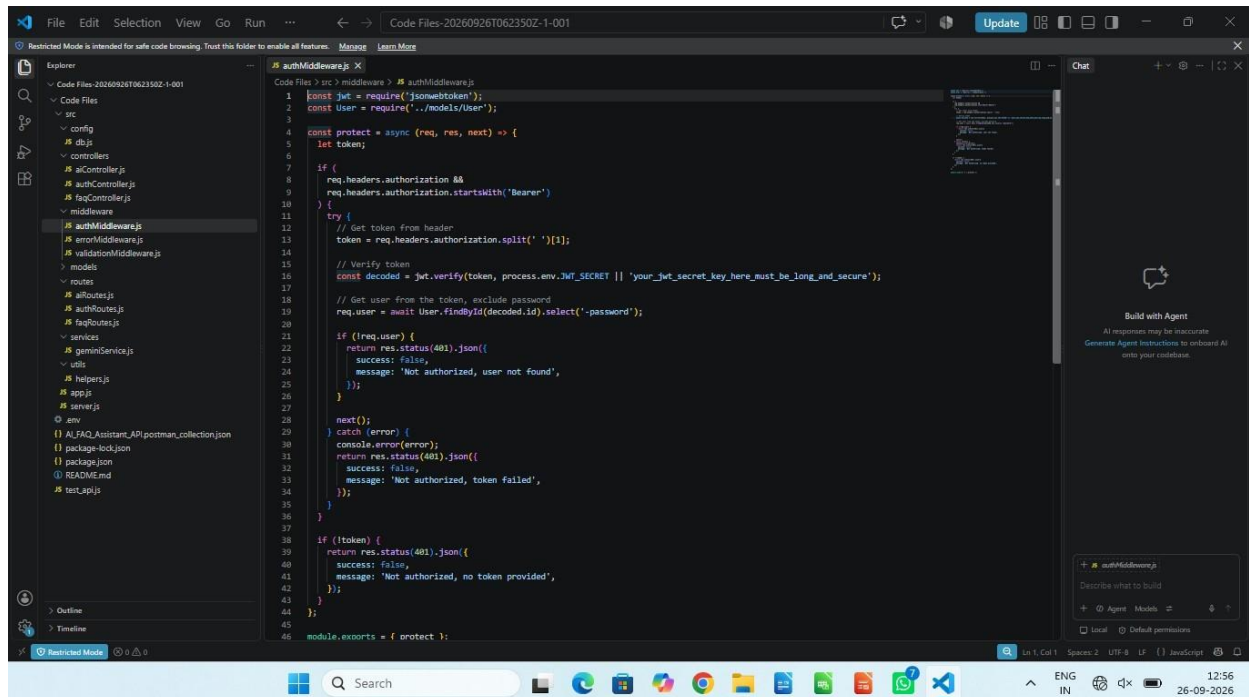


The screenshot shows a VS Code editor window with the file explorer on the left, the main editor in the center, and a chat panel on the right. The file explorer shows a project structure with folders like 'src', 'config', 'controllers', 'middleware', 'models', 'routes', 'services', 'utils', 'helpers', 'app', and 'server'. The main editor displays the 'User.js' file, which defines a Mongoose schema for a user. The schema includes fields for 'name', 'email', 'password', and 'timestamps'. The 'password' field is required, unique, lowercase, and has a minimum length of 6 characters. A pre-save hook is implemented to hash the password before saving. The 'email' field is required and has a regex validation for a valid email address. The 'timestamps' field is set to true. The chat panel on the right shows a 'Build with Agent' button and a message: 'All responses may be inaccurate. Generate Agent instructions to onboard AI onto your codebase.'

```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcrypt');
3
4 const userSchema = new mongoose.Schema({
5   name: {
6     type: String,
7     required: [true, 'Name is required'],
8     trim: true,
9   },
10   email: {
11     type: String,
12     required: [true, 'Email is required'],
13     unique: true,
14     lowercase: true,
15     trim: true,
16     match: [/^([a-zA-Z0-9_\.\-])+\@(([a-zA-Z0-9]\-)*[a-zA-Z0-9])\.[a-zA-Z]{2,4}$/i, 'Please provide a valid email address'],
17   },
18   password: {
19     type: String,
20     required: [true, 'Password is required'],
21     minlength: [6, 'Password must be at least 6 characters long'],
22   },
23 }, {
24   timestamps: true,
25 });
26
27 // Pre-save hook to hash the password before saving
28 userSchema.pre('save', async function () {
29   if (!this.isModified('password')) {
30     return;
31   }
32   const salt = await bcrypt.genSalt(10);
33   this.password = await bcrypt.hash(this.password, salt);
34 });
35
36 // Method to compare passwords
37 userSchema.methods.comparePassword = async function (candidatePassword) {
38   return bcrypt.compare(candidatePassword, this.password);
39 };
40
41 const User = mongoose.model('User', userSchema);
42 module.exports = User;
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

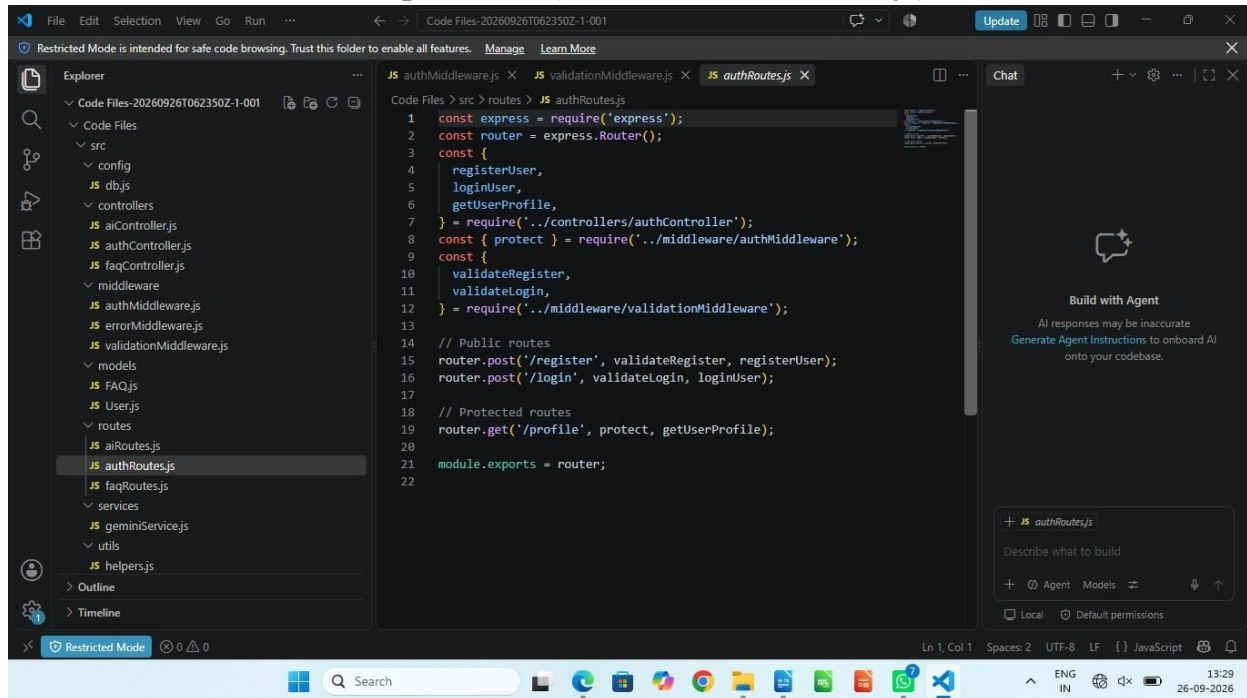


The screenshot shows a VS Code editor window with the file explorer on the left, the main editor in the center, and a chat panel on the right. The file explorer shows a project structure with folders like 'src', 'config', 'controllers', 'middleware', 'models', 'routes', 'services', 'utils', 'helpers', 'app', and 'server'. The main editor displays the 'authMiddleware.js' file, which implements a JWT authentication middleware. It checks for a 'bearer' token in the 'authorization' header, verifies the token using 'jsonwebtoken', and then finds the user in the database using the token's payload. If the user is found, it returns the user's details (excluding the password). If the token is invalid or the user is not found, it returns a 401 status with an error message. The chat panel on the right shows a 'Build with Agent' button and a message: 'All responses may be inaccurate. Generate Agent instructions to onboard AI onto your codebase.'

```
1 const jwt = require('jsonwebtoken');
2 const User = require('../models/User');
3
4 const protect = async (req, res, next) => {
5   let token;
6
7   if (req.headers.authorization && req.headers.authorization.startsWith('bearer ')) {
8     try {
9       // Get token from header
10      token = req.headers.authorization.split(' ')[1];
11
12      // Verify token
13      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_long_and_secure');
14
15      // Get user from the token, exclude password
16      req.user = await User.findById(decoded.id).select('-password');
17
18      if (!req.user) {
19        return res.status(401).json({
20          success: false,
21          message: 'Not authorized, user not found',
22        });
23      }
24
25      next();
26    } catch (error) {
27      console.error(error);
28      return res.status(401).json({
29        success: false,
30        message: 'Not authorized, token failed',
31      });
32    }
33  }
34
35  if (!token) {
36    return res.status(401).json({
37      success: false,
38      message: 'Not authorized, no token provided',
39    });
40  }
41
42  module.exports = { protect };
43 }
```

API Router Interface Endpoints

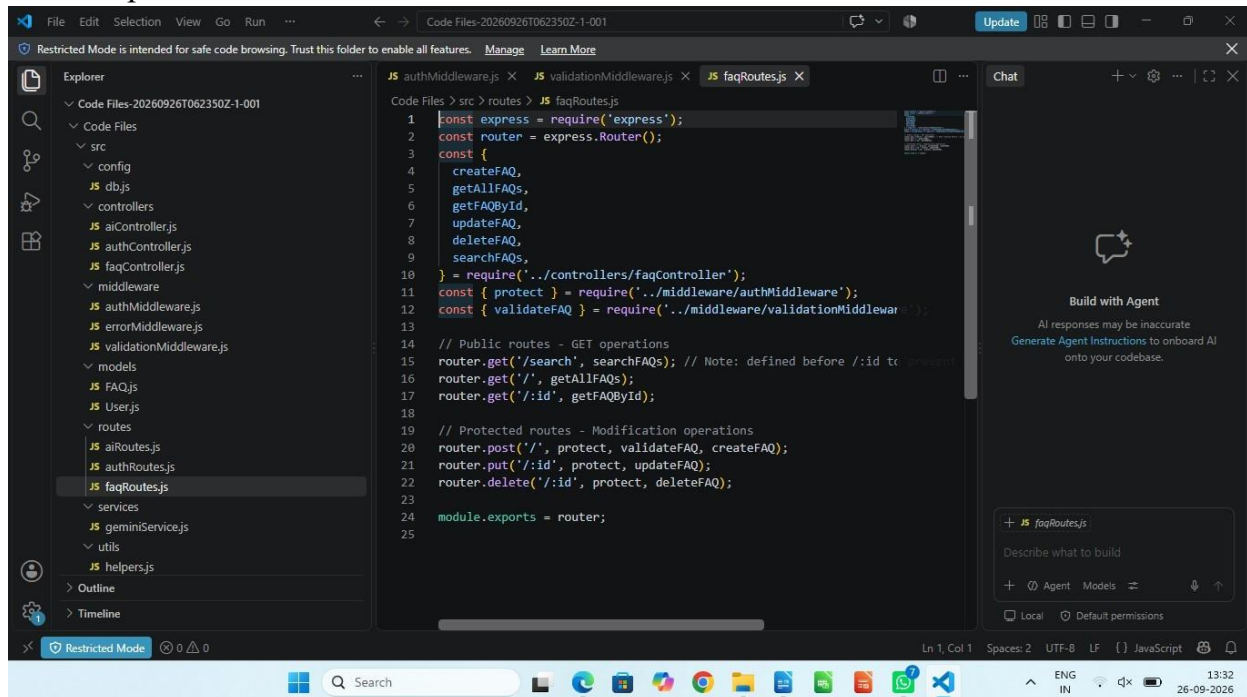
1. Public Authentication Operations (src/routes/authRoutes.js)



```
1 const express = require('express');
2 const router = express.Router();
3 const {
4   registerUser,
5   loginUser,
6   getUserProfile,
7 } = require('../controllers/authController');
8 const { protect } = require('../middleware/authMiddleware');
9 const {
10   validateRegister,
11   validateLogin,
12 } = require('../middleware/validationMiddleware');
13
14 // Public routes
15 router.post('/register', validateRegister, registerUser);
16 router.post('/login', validateLogin, loginUser);
17
18 // Protected routes
19 router.get('/profile', protect, getUserProfile);
20
21 module.exports = router;
```

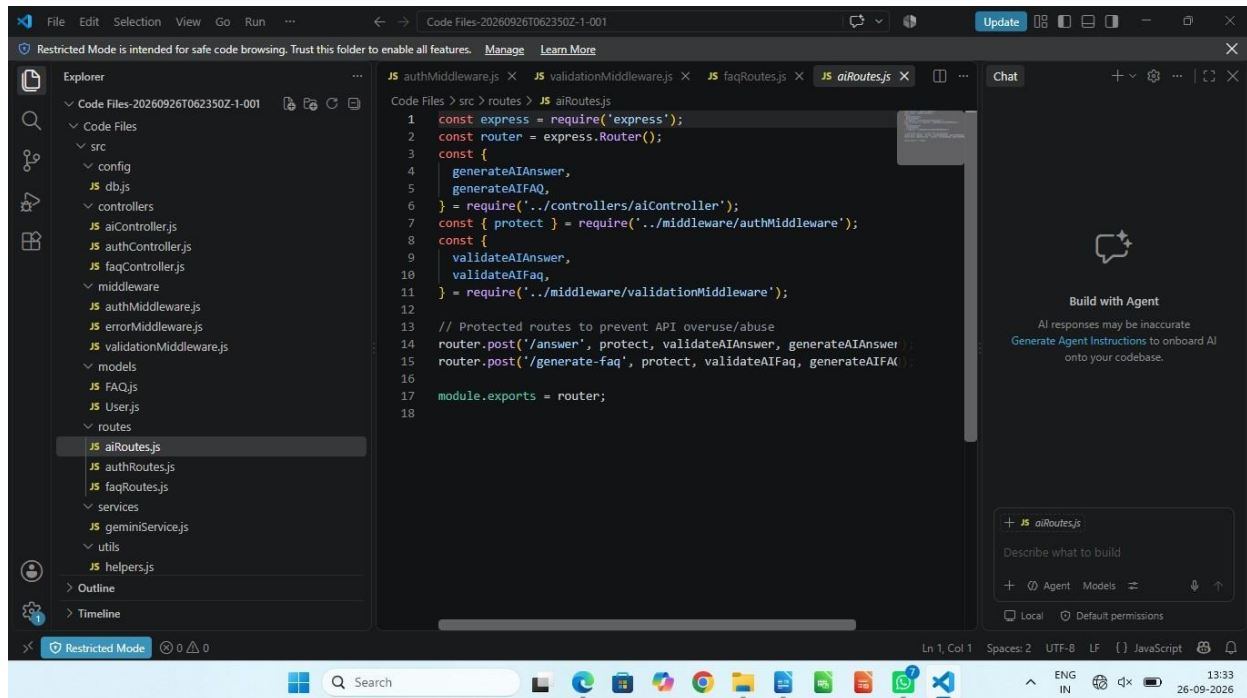
2. Protected FAQ Operations (src/routes/faqRoutes.js)

JavaScript



```
1 const express = require('express');
2 const router = express.Router();
3 const {
4   createFAQ,
5   getAllFAQs,
6   getFAQById,
7   updateFAQ,
8   deleteFAQ,
9   searchFAQs,
10 } = require('../controllers/faqController');
11 const { protect } = require('../middleware/authMiddleware');
12 const { validateFAQ } = require('../middleware/validationMiddleware');
13
14 // Public routes - GET operations
15 router.get('/search', searchFAQs); // Note: defined before /:id to prevent
16 router.get('/', getAllFAQs);
17 router.get('/:id', getFAQById);
18
19 // Protected routes - Modification operations
20 router.post('/', protect, validateFAQ, createFAQ);
21 router.put('/:id', protect, updateFAQ);
22 router.delete('/:id', protect, deleteFAQ);
23
24 module.exports = router;
```

3. AI Automation Operations (src/routes/aiRoutes.js)



Epic 5: Project Executions

Running the API Application

- Install Postman or ThunderClient (VS Code extension).
- Start the backend development server environment parameters:

Bash

❏ **npm run dev**

❏ **API Testing Specification Demo**

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

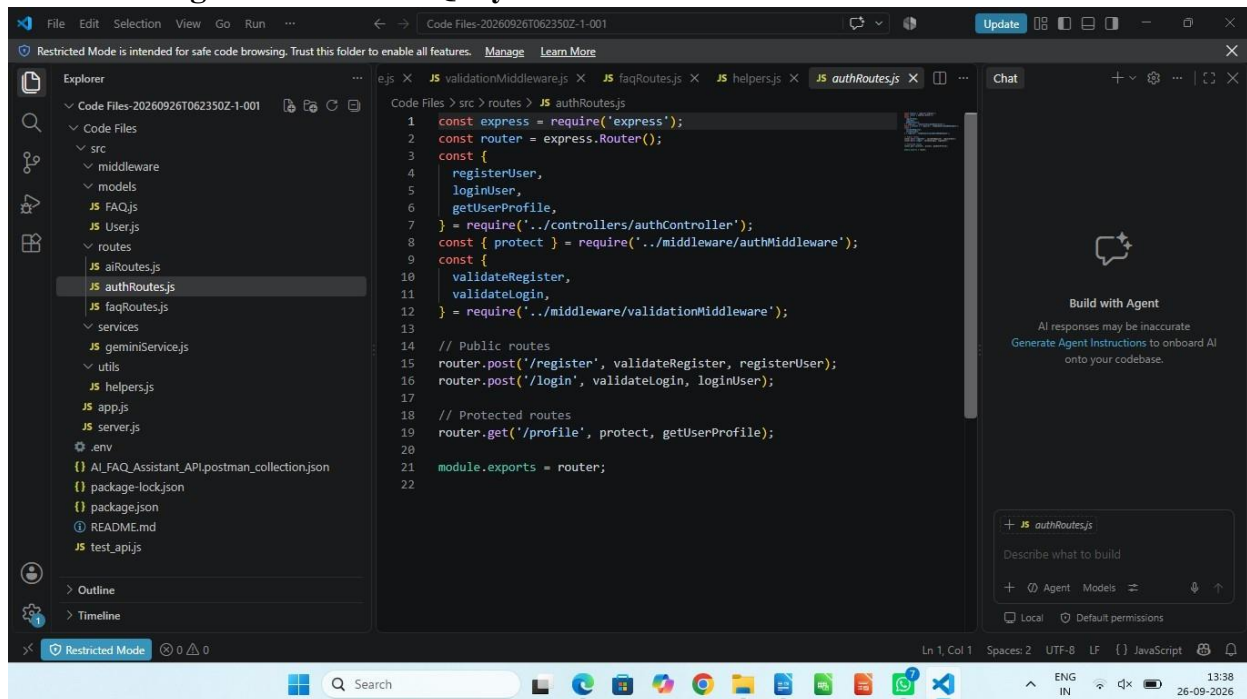
1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

JSON

```
{  
  "name": "Priya Sharma", "email": "priya@writeflow.com", "password":  
  "securepassword123"  
}
```

2. Profile Login Verification Query



- Route: POST /api/auth/login
- Access: Public
- Payload Request Body JSON:

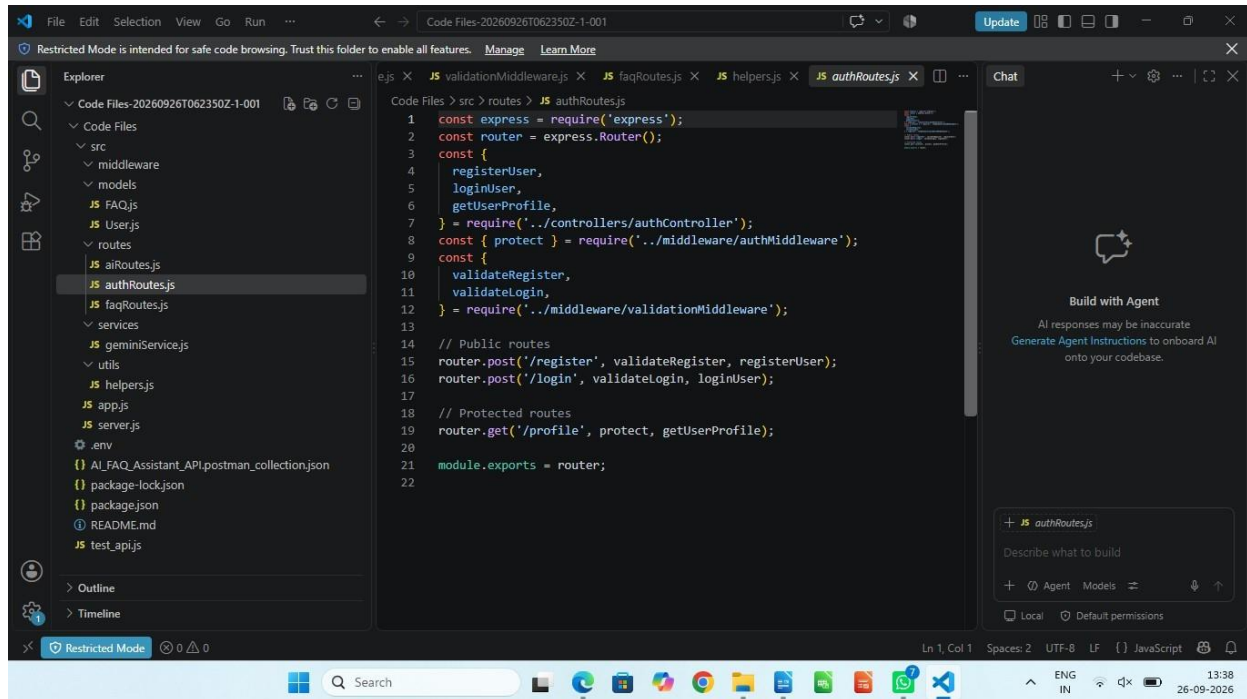
JSON

```
{  
  "email": "priya@writeflow.com", "password": "securepassword123"  
}
```

3. Resource Entity Creation (FAQ)

- Route: POST /api/faqs
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

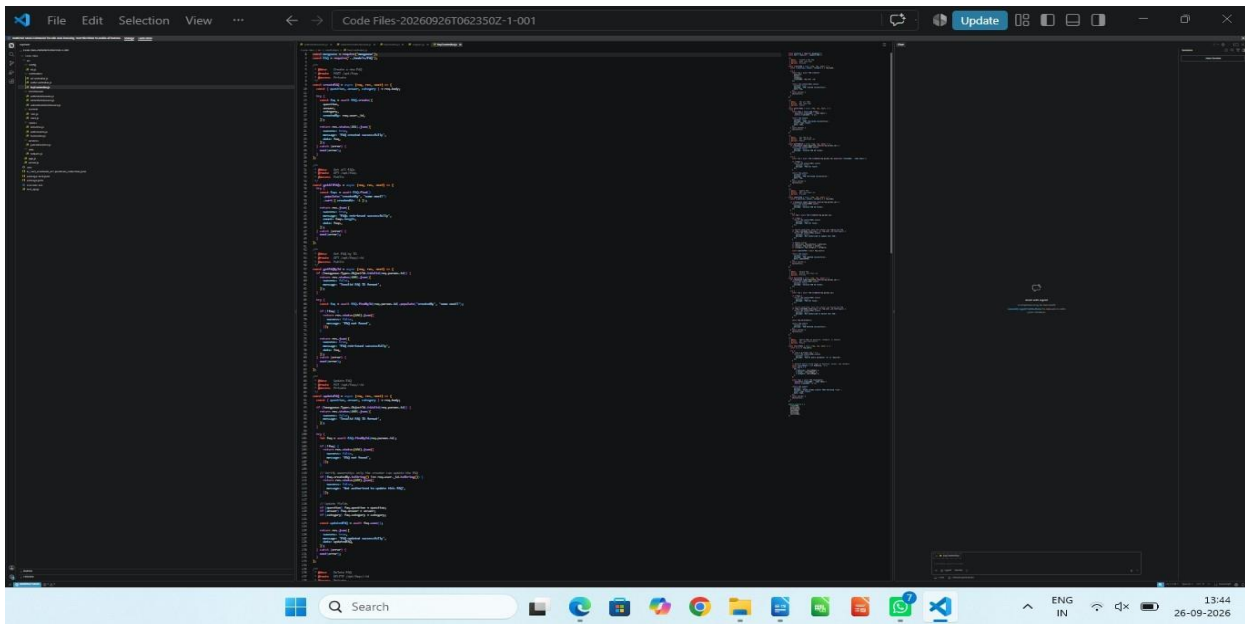


{

{

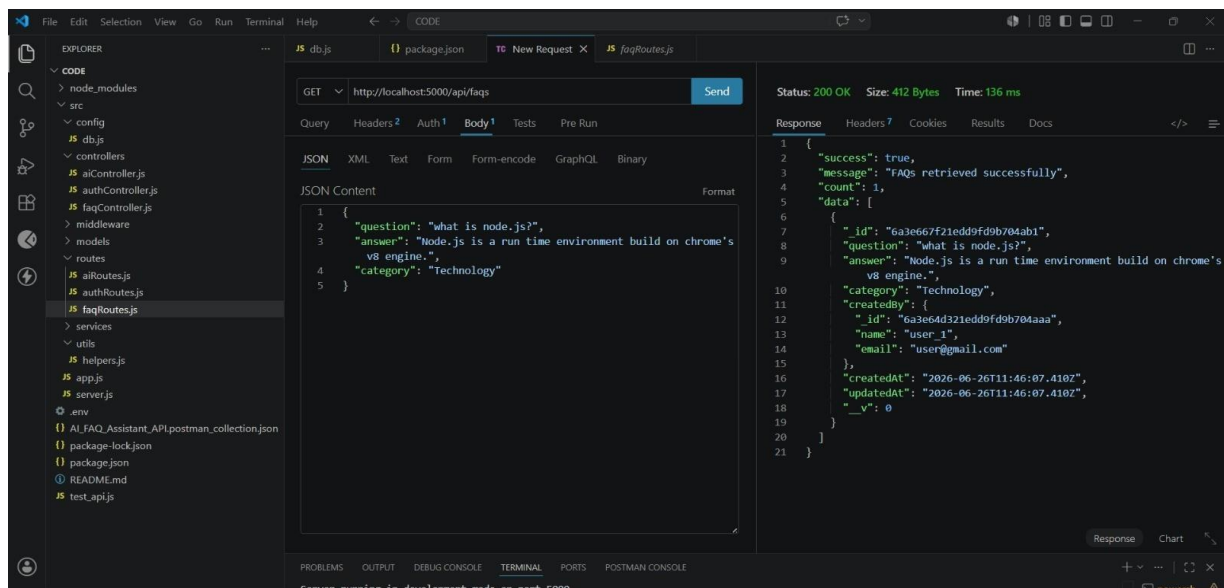
"question": "How do I configure custom category tags?", "answer": "Navigate to settings panel, append keyword tags arrays, and save schema.", "category": "Configuration"

}



4. Keyword Resource Entity Fetching (Search)

- Route: GET /api/faqs/search?q=configure
- Access: Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

- Route: POST /api/ai/generate-faq ● Access: Private (Bearer <JWT Token>) ●

Payload Request Body JSON:

JSON

```
{  
  "topic": "Mongoose schema indexing validation runtime workflow optimization"  
}
```

}

API TESTING VIDEO:

https://drive.google.com/file/d/1u_w1sigiOynhmqTDbj4NBm3CLIMunEWN/view?usp=sharing DEMO LINK:

[https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSYJ/view?](https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSYJ/view?usp=drive_link)

[usp=drive link](#)

CodeDrive:[CODE](#)